

Arquitetura e Modelagem do Estoque+

Projeto de Desenvolvimento de Negócios

Projeto	EstoqueMais
Acadêmica	Nara Silvana Moro Osio
Professor	Juliano Ramos Matos
Disciplina	Projeto de Desenvolvimento de Negócios
Data	17 de setembro de 2026

Este documento apresenta a arquitetura tecnológica e o modelo de dados do Estoque+, um aplicativo voltado ao controle simples de estoque em pequenos negócios. A solução será desenvolvida em Flutter com Dart, integrada ao Firebase, e atende às histórias prioritárias do MVP: cadastrar produtos, registrar entradas e saídas, identificar estoque baixo ou em falta e controlar datas de validade.

Modelo adotado. O Estoque+ adotará uma arquitetura cliente-servidor baseada em Backend as a Service, ou BaaS. O aplicativo desenvolvido em **Flutter com Dart** será o cliente e concentrará a interface e as validações do domínio. Os serviços gerenciados do Firebase funcionarão como servidor: Authentication para login, Cloud Firestore para persistência de dados e Storage para imagens dos produtos. Essa escolha reduz a infraestrutura necessária e é adequada ao prazo do MVP.

1.1 Camadas e responsabilidades

Camada	Responsabilidade
Apresentação	Telas Flutter criadas em Dart com widgets para cadastro, consulta, alertas e movimentações.
Aplicação	ViewModels, estados e serviços em Dart coordenam ações da interface e acesso ao Firebase.
Domínio	Regras de cadastro, entrada, saída, estoque mínimo, falta de produto e validade.
Dados	Plugins Flutter do Firebase acessam Authentication, Cloud Firestore e Storage.

1.2 Tecnologias selecionadas

Área	Tecnologia	Justificativa técnica
Aplicativo	Flutter + Dart	Permite desenvolver um aplicativo multiplataforma com uma única base de código, usando widgets modernos.
Interface	Widgets Flutter	Permitem criar telas reativas e declarativas, reduzindo código repetitivo.
Arquitetura	MVVM	Separa telas, estado e acesso a dados, facilitando testes, manutenção e evolução.
Acesso a dados	Firebase para Flutter	Integra o aplicativo Dart aos serviços gerenciados do Firebase.
Banco de dados	Firebase Cloud Firestore	Fornecer banco NoSQL em nuvem, com sincronização sem configurar servidor próprio.
Autenticação	Firebase Authentication	Gerencia login e identidade dos usuários sem servidor próprio.
Arquivos	Firebase Storage	Armazena imagens dos produtos com controle de acesso.
Hospedagem	Google Firebase	Mantém banco, autenticação e arquivos disponíveis em nuvem como serviços gerenciados.
Desenvolvimento	Android Studio ou VS Code	Ambientes compatíveis com Flutter e Dart.

Área	Tecnologia	Justificativa técnica
Versionamento	Git e GitHub	Mantêm o histórico do código e facilitam organização, recuperação de versões e colaboração.

1.3 Diagrama macro da arquitetura

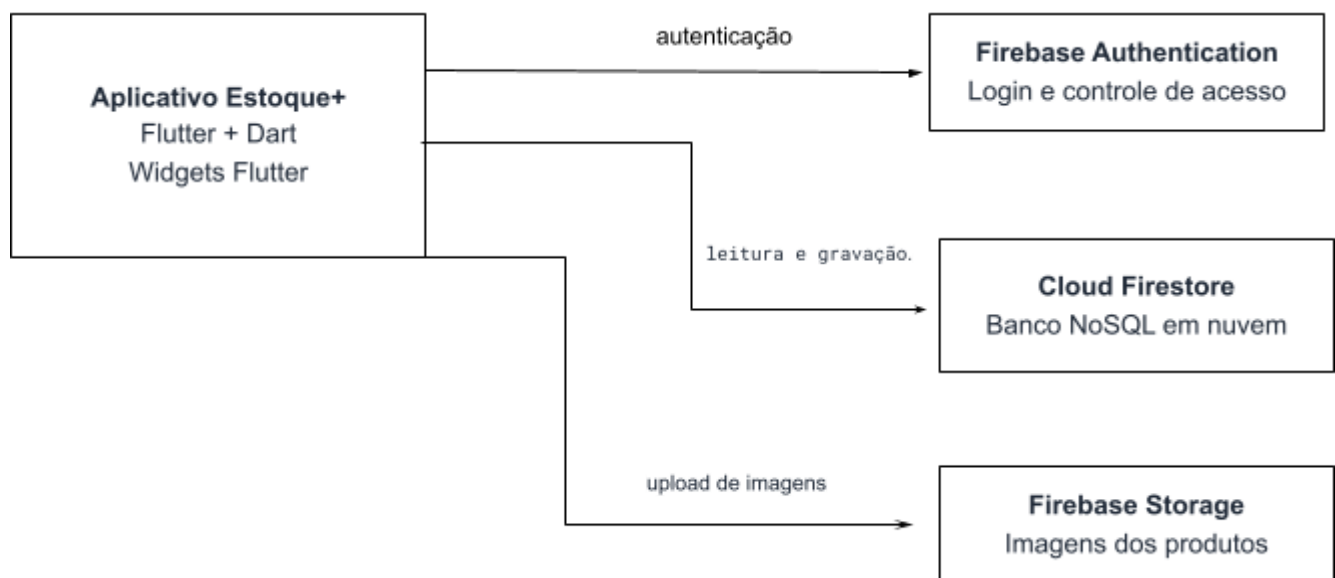


Figura 1. Comunicação entre o aplicativo Flutter e os serviços Firebase.

1.4 Comunicação entre os componentes

1. A comerciante realiza uma ação no aplicativo Estoque+ desenvolvido em Flutter com Dart.
2. O ViewModel valida os dados e chama o serviço responsável pela operação.
3. O Firebase Authentication identifica o usuário e permite aplicar regras de acesso.
4. Os plugins Firebase para Flutter leem ou gravam documentos no Cloud Firestore e enviam imagens ao Firebase Storage.
5. O estado da tela é atualizado e os widgets Flutter apresentam a confirmação ou o erro.

1.5 Segurança e configuração

A comunicação com o Firebase utiliza HTTPS. O aplicativo conterà apenas o arquivo de configuração do projeto Firebase, enquanto o acesso aos dados será protegido por Firebase Authentication e regras de segurança do Cloud Firestore e do Storage. Senhas, chaves administrativas e arquivos de conta de serviço não serão incluídos no código nem no GitHub.

2 Modelo de Dados

O modelo de dados foi derivado das histórias US01 a US04. No Cloud Firestore, Usuario, Categoria, Produto e Movimentacao serão coleções de documentos. Como o Firestore é NoSQL, as relações são mantidas por identificadores de referência lógica, equivalentes às chaves estrangeiras do modelo conceitual.

2.1 Modelo lógico equivalente ao DER

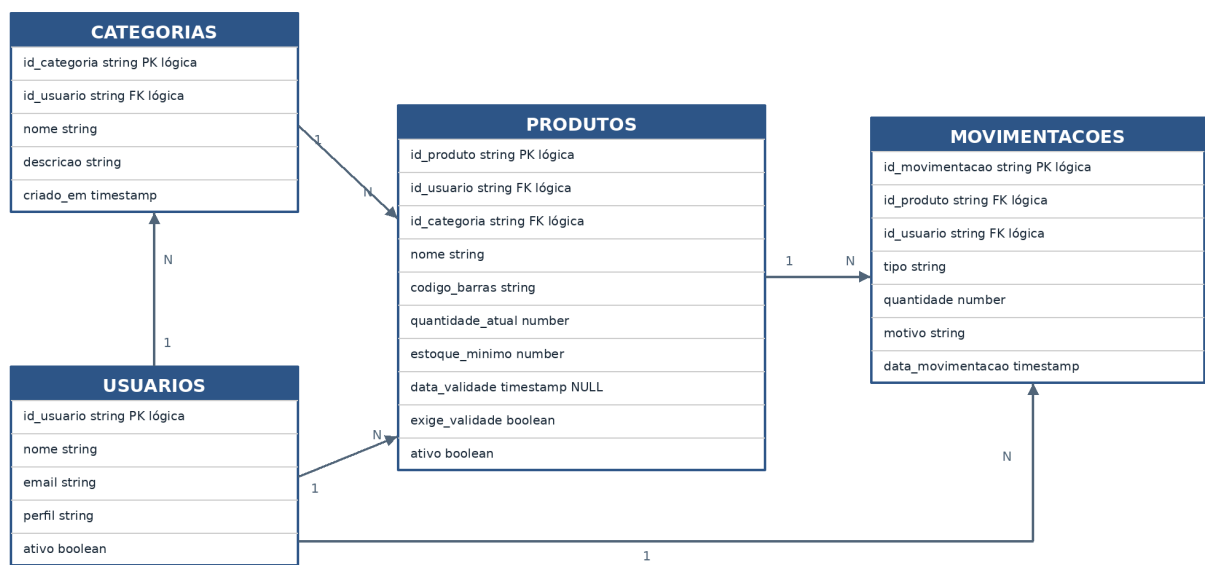


Figura 2. Modelo das coleções e referências lógicas do Estoque+ no Firestore.

2.2 Entidades e atributos

Entidade	Atributo	Chave	Finalidade
Usuario	id_usuario	PK lógica	ID único, relacionado ao usuário autenticado no Firebase Authentication.
Usuario	nome	-	Nome da pessoa que utiliza o sistema.
Usuario	email	-	E-mail usado para identificação e login.

Entidade	Atributo	Chave	Finalidade
Usuario	perfil	-	Define as permissões do usuário no aplicativo.
Usuario	ativo	-	Indica se o acesso ao sistema está habilitado.
Categoria	id_categoria	PK lógica	ID único do documento da categoria.
Categoria	id_usuario	FK lógica	Define o usuário proprietário da categoria.
Categoria	nome	-	Nome usado para organizar os produtos.
Categoria	descricao	-	Informação opcional sobre a categoria.
Categoria	criado_em	-	Data e hora de criação do registro.
Produto	id_produto	PK lógica	ID único do documento do produto.
Produto	id_usuario	FK lógica	Define o usuário proprietário do produto.
Produto	id_categoria	FK lógica	Referência lógica ao documento Categoria.id_categoria.
Produto	nome	-	Nome do produto.
Produto	codigo_barras	-	Código usado para localizar o produto.
Produto	quantidade_atual	-	Saldo disponível no estoque.
Produto	estoque_minimo	-	Limite usado para gerar alerta de estoque baixo.
Produto	data_validade	-	Validade do produto, quando aplicável.
Produto	exige_validade	-	Indica se a validade é obrigatória.
Produto	ativo	-	Permite desativar sem apagar o histórico.
Produto	criado_em	-	Data e hora de criação do cadastro.
Movimentacao	id_movimentacao	PK lógica	ID único do documento da movimentação.

Entidade	Atributo	Chave	Finalidade
Movimentacao	id_produto	FK lógica	Referência lógica ao documento Produto.id_produto.
Movimentacao	id_usuario	FK lógica	Referência lógica ao usuário que realizou a operação.
Movimentacao	tipo	-	Define se a operação é ENTRADA ou SAIDA.
Movimentacao	quantidade	-	Quantidade movimentada.
Movimentacao	motivo	-	Observação ou justificativa da movimentação.
Movimentacao	data_movimentacao	-	Data e hora da entrada ou saída.

2.3 Relacionamentos e cardinalidades

Relacionamento	Cardinalidade	Interpretação
Categoria e Produto	1:N	Uma categoria pode classificar nenhum ou vários produtos; cada produto pertence a uma categoria.
Produto e Movimentação	1:N	Um produto pode possuir nenhuma ou várias movimentações; cada movimentação pertence a um produto.
Usuário e Categoria	1:N	Um usuário pode possuir nenhuma ou várias categorias; cada categoria pertence a um usuário.
Usuário e Produto	1:N	Um usuário pode possuir nenhum ou vários produtos; cada produto pertence a um usuário.
Usuário e Movimentação	1:N	Um usuário pode realizar nenhuma ou várias movimentações; cada movimentação registra um usuário responsável.

2.4 Regras de integridade do domínio

- O nome do produto é obrigatório.
- A quantidade atual, o estoque mínimo e a quantidade movimentada não podem ser negativas.
- Uma movimentação de saída não pode superar a quantidade disponível.

- Quando exige_validade for verdadeiro, a data de validade deverá ser informada.
- Quantidade igual a zero significa produto em falta.
- Quantidade maior que zero e menor ou igual ao estoque mínimo significa estoque baixo.
- A movimentação deve ser registrada junto com a atualização do saldo na mesma transação, evitando divergência entre histórico e quantidade atual.
- Produtos que vencem em até 30 dias devem gerar alerta.
- Produtos e movimentações não são apagados; os produtos são desativados pelo campo ativo.

3 Diagrama UML Complementar

O diagrama de atividades foi escolhido porque a saída de estoque envolve uma decisão crítica: verificar se existe quantidade suficiente antes de alterar o saldo. Esse fluxo esclarece os caminhos de sucesso e de rejeição previstos no critério de aceite principal da US02.

3.1 Atividades do registro de saída

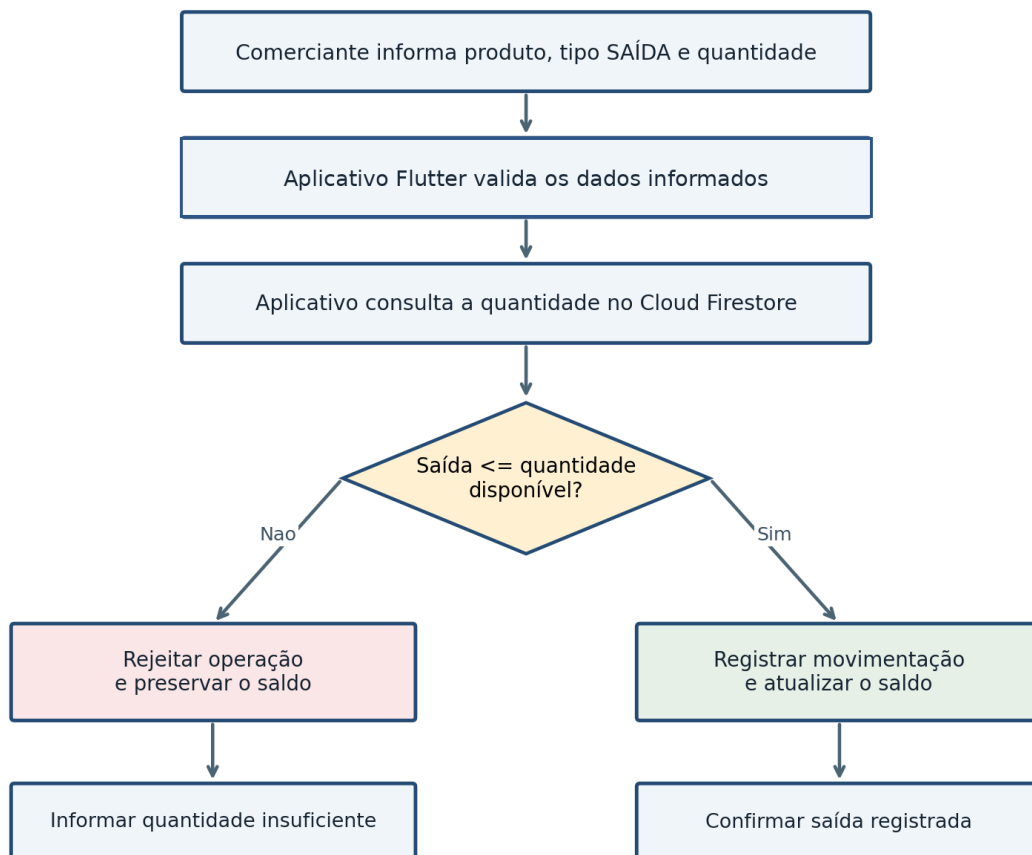


Figura 3. Diagrama de atividades da validação e do registro de saída.

3.2 Resultado esperado

Se houver quantidade suficiente, o aplicativo Flutter registra a movimentação e atualiza o saldo no Cloud Firestore. Se a saída for maior que o estoque disponível, a operação é recusada e nenhum

dado é alterado. Assim, o aplicativo mantém a quantidade confiável e informa claramente o motivo da rejeição.